

Agile Estimator v2

Relatorio de Entrega - Marcos 4 e 5

Responsavel tecnico: Savio Mendes

Data do relatorio: 11/05/2026

API em producao: <https://agile-estimator-ofc.onrender.com>

Board Trello demo: <https://trello.com/b/DKf6KNh2/testeagileestimator>

Escopo deste documento

Este documento confirma a entrega completa dos Marcos 4 e 5 do contrato Fase 2 - Agile Estimator v2. Cobre o que foi entregue, como foi implementado, bugs corrigidos, artefatos gerados e instrucoes de operacao.

Marco 4 - Infraestrutura e Deploy

Campo	Detalhe
Data de entrega contratada	16/04/2026
Valor do marco	R\$ 460,00
Status	ENTREGUE

4.1 Objetivo do Marco

Containerizar a API de inferencia com Docker e implanta-la em infraestrutura de nuvem com URL publica acessivel via HTTPS, garantindo reinicio automatico em caso de falha.

4.2 O Que Foi Entregue

Entregavel	Descricao	Status
Dockerfile	Imagem python:3.11-slim; uvicorn; porta configuravel via \$PORT	Entregue
docker-compose.yml	restart: always - reinicio automatico em falha	Entregue
Deploy Render.com	API publicada em URL publica com HTTPS automatico	Entregue
GET /health	Verifica modelo carregado, arquivo .pkl e inferencia de sanidade	Entregue
POST /predict	Endpoint de inferencia com schema Pydantic validado	Entregue
Logging estruturado	Logs de predicao e erros via app/logger.py	Entregue

4.3 Dockerfile

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

4.4 docker-compose.yml (reinicio automatico)

```
version: "3.9"
services:
  api:
    build: .
    container_name: agile_api
    ports:
      - "8000:10000"
    volumes:
      - ./app
    restart: always      # reinicia automaticamente em falha
```

4.5 Endpoints em Producao

Endpoint	Metodo	Descricao
https://agile-estimator-ofc.onrender.com/predict	POST	Inferencia - retorna horas-pessoa

https://agile-estimator-ofc.onrender.com/health	GET	Health check com 3 verificacoes
https://agile-estimator-ofc.onrender.com/docs	GET	Swagger UI (FastAPI automatico)

4.6 Health Check (/health)

O endpoint realiza 3 verificacoes a cada chamada:

- Modelo carregado em memoria (model is not None)
- Arquivo agile_estimator_v2.pkl existe no filesystem do container
- Inferencia de sanidade - executa predict() e verifica resultado valido

Retorna {"status": "ok"} quando todos os checks passam, "degraded" se algum falha.

4.7 Plataforma de Deploy - Render.com

- Deploy automatico via push para branch main no GitHub
- HTTPS nativo com certificado SSL automatico
- Reinicio automatico em caso de crash (Docker restart: always)
- Cold start de ~30s se inativo ha mais de 15 minutos (plano gratuito)
- Para eliminar cold start: upgrade para plano pago (\$7/mes) ou ping periodico via cron

Marco 5 - Integracao Streamlit e Documentacao Tecnica

Campo	Detalhe
Escopo	Integracao Streamlit com API REST + Documentacao + Testes
Status	ENTREGUE

5.1 Objetivo do Marco

Adequar a interface Streamlit para consumir exclusivamente a API REST, removendo qualquer execucao direta do modelo no frontend. Documentar a arquitetura, o pipeline de dados, o processo de re-treino manual e as instrucoes de operacao. Realizar testes e validacoes finais.

5.2 Integracao Streamlit - API REST (sem modelo local)

A interface nao executa o modelo localmente. Todo processamento de inferencia ocorre na API hospedada no Render. O Streamlit realiza somente pre-processamento client-side:

Etapas	Onde Ocorre	Descricao
Coleta de inputs	Streamlit	Manual / CSV / Trello
Padronizacao function_points	Streamlit	(fp - 514.86) / 516.24 via scaler_maxx.pkl[0]
StandardScaler + PCA	Streamlit	4 fatores Likert -> PC1, PC2 via artefatos .pkl
POST /predict	Streamlit -> API REST	Envia {function_points_std, PC1, PC2}
Inferencia Lasso	API no Render	model.predict(X) + np.exp() -> effort_hours
Resultados	Streamlit	Tabela, graficos Altair, download CSV

5.3 Funcionalidades da Interface

Funcionalidade	Descricao
Tutorial interativo	Explica modelo, inputs, outputs, escala Likert e limitacoes
Input Manual	Formulario com 5 campos; validacao Likert 1-5; nao sobreescreve dados anteriores
Calculadora IFPUG embutida	Calcula AFP via pesos EI/EO/EQ/ILF/EIF; ajuste VAF opcional
Upload CSV	Importacao em lote; preview 10 linhas; botao de confirmacao antes de carregar
Integracao Trello	Importa de board publico via URL; sem autenticacao; preview antes de carregar
Calculo de estimativas	Chama API individualmente para cada projeto; dias e semanas estimados
Visualizacoes (Altair)	Histograma, scatter FP x esforco, scatter complexidade x esforco, bar chart
Download CSV	Exporta project_id, horas, dias e semanas estimados
FAQ lateral (sidebar)	Perguntas frequentes sobre o sistema e o modelo

5.4 Integracao Trello (entrega adicional - Phase 3 antecipada)

A integracao Trello estava prevista para a Phase 3 (escopo futuro), mas foi antecipada e entregue nesta fase sem custo adicional.

- Aceita URL no formato https://trello.com/b/<board_id>
- GET ao endpoint publico https://trello.com/b/<id>.json (sem autenticacao)
- Mapeamento case-insensitive de nomes de campos com tolerancia a erros ortograficos
- Preview dos dados antes de carregar (evita sobreescrita acidental)
- Validacao de campos obrigatorios com mensagem de erro descritiva

Pipeline de Dados e Modelo

Modelo de Machine Learning

Atributo	Valor
Algoritmo	Lasso Regression (sklearn.linear_model.Lasso)
Selecao	Menor RMSE entre Ridge, Lasso, SVR, RandomForest, GradientBoosting
Validacao cruzada	RepeatedKFold(n_splits=5, n_repeats=10, random_state=42)
Hiperparametro alpha	0.1 (selecionado por GridSearchCV - grid: [0.001, 0.01, 0.1, 1])
RMSE no holdout	~1.877 horas
MAPE no holdout	~49%
Dataset de treino	Maxwell - 57 projetos (62 menos outliers por IQR em effort_hours)
Variavel-alvo	log(effort_hours); API retorna np.exp(prediction)
Coeficientes Lasso	function_points: 0.341 PC1: 0.167 PC2: -0.272
Intercept	8.358 => np.exp(8.358) ~ 4.254 horas (media do dataset)

Pipeline de Pre-processamento de Inferencia

```
# 1. Padronizar function_points (artefato: scaler_maxx.pkl, indice 0)
fp_std = (function_points - 514.8596) / 516.2373

# 2. Padronizar 4 fatores Likert (artefato: scaler_pca_features.pkl)
X_scaled = scaler_pca_features.transform([[perf_req, complex_proc, inst_ease, add_complexity]])

# 3. Reducao de dimensionalidade (artefato: pca_2.pkl, ~70% variancia explicada)
PC1, PC2 = pca_2.transform(X_scaled)[0]

# 4. POST /predict
payload = {"function_points": fp_std, "PC1": PC1, "PC2": PC2}

# 5. API retorna
effort_hours = np.exp(model.predict([[fp_std, PC1, PC2]])[0])
```

Como Re-treinar o Modelo (processo manual)

1. Passo 1 - Dados
- Atualizar model/data/raw/maxx.csv com novos projetos, se houver.
2. Passo 2 - EDA
- Executar model/notebooks/inferencia/agile_v2/eda.ipynb para re-validar correlacoes, outliers e PCA.
3. Passo 3 - Treino
- Executar model/notebooks/treino/agile_v2/modelagem-ml.ipynb. O notebook salva automaticamente os artefatos em api/artifacts/ e model/artifacts/.
4. Passo 4 - Validar
- Verificar que os coeficientes Lasso sao nao-zero. O notebook foi corrigido para usar grid.best_estimator_ (evita refit com alpha=1.0 padrao).
5. Passo 5 - Deploy
- git push para branch main. O Render detecta alteracao nos .pkl e refaz o deploy automaticamente em ~2 minutos.

Testes e Validacoes Finais

Testes de Integracao Realizados

Cenario de Teste	Resultado Esperado	Status
POST /predict com fp=500, fatores=3,3,3,3	~3.000-5.000h; nao identico a outros inputs	OK
POST /predict: fp=200 vs fp=1000 (mesmos fatores)	Predicoes diferentes proporcionais ao fp	OK
POST /predict: fatores 1,1,5,1 vs 5,5,1,5	Predicoes diferentes por variacao Likert	OK
GET /health	Status: ok; inference_ok: true	OK
Adicionar 3 projetos manuais diferentes	Todos listados; estimativas distintas	OK
Upload CSV com 3 projetos	Todos estimados individualmente	OK
Importar board Trello publico	Cartoes com campos preenchidos importados	OK
Adicionar projeto apos importar Trello	Nao sobreescreve dados do Trello	OK
Botao Limpar dados	Zera st.session_state.data; lista some	OK
Download CSV de resultados	Arquivo com horas, dias e semanas exportado	OK

Bugs Identificados e Corrigidos

Bug	Causa Raiz	Correcao Aplicada
Todas predicoes identicas (~3981h)	Notebook salvava modelo com alpha=1.0 (coef=[0	Artefato substituido pelo modelo correto; notebook corr
function_points sem efeito na predi	fp passado raw para API; modelo foi treinado com	Adicionado (fp - 514.86) / 516.24 em preprocess_for_a
Botao 'Adicionar Projeto' nao funcio	Ausencia de st.rerun() apos append no session_st	Adicionado st.toast() + st.rerun() no callback do botao
Importacao Trello sobreescrevia da	Fetch rodava em todo rerun do Streamlit, nao so n	Fetch movido para dentro do callback do botao 'Buscar
CSV sobreescrevia dados em todo	Mesmo padrao do Trello: escrita fora de callback d	Adicionado botao 'Carregar este CSV' com preview an
Botoes 'Limpar dados' nao funciona	Ausencia de st.rerun() apos limpeza do session_st	Adicionado st.rerun() nos callbacks de limpeza

Inventario de Artefatos Entregues

Arquivo / Artefato	Tipo	Descricao
api/artifacts/model/agile_estimator_v2.pkl	Lasso Pipeline	alpha=0.1; coef=[0.341, 0.167, -0.272]
api/artifacts/preprocessing/pca_2.pkl	PCA n=2	~70% variancia; PC1=complexidade tecnica
api/artifacts/preprocessing/scaler_pca_features.pkl	StandardScaler	4 fatores Maxwell (T03,T09,T11,T15)
api/artifacts/preprocessing/scaler_maxx.pkl	StandardScaler	Indice 0 = function_points (mean=514.86)
api/Dockerfile	Docker	python:3.11-slim; porta via \$PORT
api/docker-compose.yml	Docker Compose	restart: always
api/app/main.py	FastAPI	Rotas /predict e /health; CORS
api/app/predict.py	Python	Logica de inferencia; feature selection
api/app/schemas.py	Pydantic	PredictionInput: function_points, PC1, PC2
streamlit_app/app.py	Streamlit	Interface completa; 4 modos entrada; 3 abas resultados
streamlit_app/scripts_app/get_public_trello_board.py	Python	Importador Trello publico com alias de campos
model/notebooks/treino/agile_v2/modelagem-ml.ipynb	Jupyter	Pipeline treino documentado e corrigido
model/notebooks/inferencia/agile_v2/eda.ipynb	Jupyter	EDA com decisoes de modelagem documentadas
docs/TECHNICAL_DOCUMENTATION.md	Markdown	Documentacao tecnica completa e atualizada
docs/BUSINESS_DOCUMENTATION.md	Markdown	Documentacao de negocio e proposta comercial

Instrucoes de Operacao

Rodar Localmente

```
# API
cd api
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
# Swagger: http://localhost:8000/docs

# Interface Streamlit (terminal separado)
cd streamlit_app
streamlit run app.py
# Para API local: altere API_URL em app.py para http://localhost:8000/predict

# Via Docker
cd api && docker-compose up --build
```

Testar a API Diretamente

```
# Health check
curl https://agile-estimator-ofc.onrender.com/health

# Predicao (function_points ja padronizado: (500 - 514.86) / 516.24 = -0.029)
curl -X POST https://agile-estimator-ofc.onrender.com/predict \
  -H "Content-Type: application/json" \
  -d '{"function_points": -0.029, "PC1": 0.5, "PC2": -0.3}'

# Resposta esperada: {"prediction": 4982.3}
```

Observações Importantes

- Cold start da API: primeira requisição após 15 min de inatividade leva ~30s. Chamar GET /health antes de estimar reduz o impacto.
- O modelo estima o esforço do PROJETO INTEIRO (não por sprint). Para 500 FP e complexidade média, espera-se ~3.000-5.000 horas-pessoa.
- Installation Ease é inversamente proporcional ao esforço: valor 5 = muito fácil de instalar (menor esforço nessa dimensão). Valor 1 = muito difícil.
- MAPE ~49%: use as estimativas como baseline e para comparação relativa entre projetos, não como compromisso contratual absoluto.
- Re-treino do modelo: processo manual descrito na seção anterior. Não requer intervenção na API durante o treino.

Confirmacao de Entrega

Este documento comprova a entrega completa dos Marcos 4 e 5 do contrato Fase 2 - Agile Estimator v2, conforme acordado por email.

Marco	Descricao Contratada	Data Prevista	Status	Valor
Marco 4	Infraestrutura e Deploy (Docker + Render)	16/04/2026	ENTREGUE	R\$ 460,00
Marco 5	Integracao Streamlit + Documentacao Tecnica	Fase 2	ENTREGUE	Incluso

Todos os entregaveis listados neste documento foram implementados, testados e estao disponiveis no repositorio do projeto e em producao na URL publica <https://agile-estimator-ofc.onrender.com>.

Responsavel Tecnico: Savio Mendes
11/05/2026

Cliente / Contratante
Data: __/__/____